

Phase 3 — Amendment 3

P3.8 extended — who an answer returns, and what it shows for each row

Status: proposed, 8 October 2026. Raised after the Phase 2 checkpoint; no Phase 2 definition of done covers it.

The request

Two faults in the assistant, raised together:

1. The chat returns at most **12 people** for any query. The cap applies both to “who works in finance” and to recommendation-style questions. Remove it, and return everyone who qualifies.
2. Attach **confidence and a rationale** to each person returned.

Both are right in substance. Both need care in form, and the reasons belong here because they will be proposed again.

Why this is P3.8 and not a chat fix

P3.8 is retrieval. The cap, the qualifying test and the per-row explanation are all answers to “which people come back from a query, and what is shown for each” — the same question P3.8 already asks about query expansion.

It is also the question the **API contract** has to answer. The scope revision puts the API over the shared build function with a written contract, and the MCP server thin over the API. If the chat decides retrieval in its own panel, the API decides it again, and the two drift. Deciding it once in P3.8 means the dashboard, the API and the MCP server return the same people for the same query.

1. The cap

The cap is a count where the answer should be a judgement about fit. Returning everyone who qualifies is the same mistake at larger scale: a list produced by a rule rather than by relevance. Four people is a complete answer to “who works in finance” when four work in finance. Twelve padded to twelve is a falsehood by inclusion, the same class as a confident wrong classification, arriving in the answer rather than on a node.

No target count in either direction. No floor, no ceiling, and no stated total standing in for relevance.

The reply budget. Today the cap also keeps the model’s reply short. Every question sends the whole network (about 9,000 tokens at 442 people, design record §6.6), and the reply shares an 8,000-token output budget with the model’s adaptive thinking (D-42, D-45). A cut-off reply is refused rather than shown. Without a ceiling, a genuinely broad answer (sixty people in finance) must still fit. So the model returns only the people it selects, by reference, and code renders each row from the record. The reply then stays small however large the answer is.

The cost consequence is real and must be measured, not assumed. The current figure, about \$0.02 a question, is an estimate (design record §6.6), not a measurement. Under the shape below, the model is sent the candidate set rather than the whole network, which is cheaper for a narrow query and no cheaper for a broad one. The daily ledger (D-81) now actually enforces, so measure before shipping, with the same discipline D-70’s follow-up required.

2. Code narrows; the model selects

Why one stage cannot do it. A deterministic code predicate cannot judge fit for most questions. “Who works in finance” has a mechanical test: the occupation group. “Who might know about scaling a small analytics team” does not. It needs the titles read and judged. Yet a model choosing freely from the whole network cannot be a retrieval contract, because D-69 and D-84 both measured it as non-deterministic. The two pull against each other. The shape below resolves that by splitting the work, not by assertion.

Stage 1: code produces a candidate set from what the records hold: occupation group, title, user corrections, notes, and P3.8’s expanded terms. It is deterministic and reproducible, and deliberately wide: it errs toward including. For each candidate it records the field and the term that put them in the set.

Stage 2: the model selects from that set. The selection is a judgement and cannot be a code predicate. The model names only people already in the candidate set. Code checks this, and drops any person named outside it, so the model can narrow but never add.

The candidate set and P3.8’s query expansion are one function, not two. Both answer “which people match this query”. Building them separately would be DoD 11’s failure one level down: two implementations that agree until they don’t.

The label filter is part of stage 1, and it is not a confidence interval. The request was “only people who fall within a particular confidence interval, say 0.75–1.0”.

Those numbers are not probabilities. They come from CONFIDENCE_SCORES, a fixed map from the classifier’s own *high* / *medium* / *low* label: *high* is 0.95, *medium* 0.75. They are a relabelling of three words, not a calibrated estimate of being correct.

And they are not stable. D-84 measured six runs on one identical title and About text through the single-call path: the About-informed reading gave 0.95, 0.75, 0.75 and three abstentions. The same input, the same model, the same configuration, three different scores. D-69 measured the same instability on the batched path at the level of the assigned group.

So a band of 0.75–1.0 means **“everyone the classifier labelled high or medium on the draw that built this network”**. That is a perfectly reasonable filter. It is not a confidence interval, and calling one on screen would be the exact class of claim D-69, D-70 and D-84 were each logged for.

What ships in stage 1:

- The filter is on the **label**, stated in words: people the classifier placed with high or medium confidence. No numeric band on screen.
- People with no group — those needing review, and those not classified yet — are excluded, and **not** silently. The answer says how many were left out for that reason. This is a change. Today the prompt’s rule 2 lets the chat include a person needing review when their title is relevant, marked as unreviewed and never given a group, and rule 2b does the same for people not classified yet.
- A user’s own correction outranks any classifier label. A corrected person qualifies regardless of what the classifier said.

Wide across fields, strict about placement. Excluding people with no group pulls against stage 1 being deliberately wide, and the tension was considered and decided. Stage 1 is wide across its sources (title, group, corrections, notes, expanded terms), because widening the sources is how a genuine match is not missed. It is strict about placement, because admitting unplaced people is how a non-match gets presented as one. Admitted with an unreviewed marker, they would sometimes be selected, and the marker would be doing the abstention rule’s work, and markers lose to lists. DoD 9 states how many were excluded. Anyone who wants them has the review queue, where an unplaced person can be acted on rather than merely listed.

3. The per-row explanation

“Rationale” has a permitted form and an impermissible one, and the line is already drawn in Amendment 2.

The cross-cutting rule: no feature attributes a skill, tool or capability to a named individual unless that individual’s own words are its source.

So the explanation cannot say *why this person is a good fit* — that is an attributed capability, from the user’s note or the model’s inference, neither of which is the person’s own words.

It can say what matched, which is Amendment 2’s own DoD criterion 2: the result list shows which term caused each row to match. Today the chat already attaches a reason to each row: the model’s own **why**, shown after a dash and limited by the prompt’s rule 3. This replaces that generated sentence. Extended here, each returned person carries what stage 1 recorded, not anything the model writes:

- the field that matched — their job title, their occupation group, or a note the user wrote themselves
- the term that matched it, including an expanded term where P3.8 widened the query
- their classification state: placed by the classifier, corrected by the user, or not placed

That is attribution to the data, not to the person. It is also more useful: a user who can see *why* a row is in the list can tell a good match from a bad one, which a sentence of generated justification does not allow.

Ranking follows the same rule. Code orders the selected rows — corrected first, then high, then medium — and names the ordering. A numeric score per person, implying a measured likelihood of fit, is not shipped.

Definition of done

These extend Amendment 2’s six, which stand unchanged.

7. **No target count.** No floor and no ceiling. The answer is the people the selection judged to fit, however many that is, and never padded to a number.
8. **Code narrows; the model selects.** Stage 1 filters on the classifier’s label or the user’s correction, never on a numeric threshold presented as confidence. Stage 2 selects only from stage 1’s set, and code drops any person the model names outside it.
9. **Exclusions are stated.** The answer says how many people were left out for being unplaced, rather than silently omitting them.
10. **Each row says what matched** — field, term, and classification state, as stage 1 recorded them — and nothing about the person’s capability.
11. **One retrieval contract.** The dashboard, the API and the MCP server derive the same candidate set from the same query, from one implementation, which is also P3.8’s query expansion. The selection over that set is a model judgement and is not guaranteed identical between calls. D-69 and D-84 both measured the model as non-deterministic, and a contract promising identical output would be unmeetable by construction. The contract guarantees the candidate set, and that every selected person is in it, and states plainly that the selection varies.
12. **Cost is measured, not assumed.** The per-question cost of an uncapped answer is measured on a real network before this ships, and the free-tier allowance is revisited against that figure.

Pre-registered gate

Amendment 2’s gate stands: at least 15 of 20 related pairs match, at most 2 of 20 unrelated pairs match, on a probe set written before any expansion code runs.

It is extended, before implementation, with three cases that the two-stage answer introduces:

- **No unplaced person appears as placed.** On a probe query against a network with known unplaced people, zero unplaced people appear in the returned list, and the stated exclusion count equals the number excluded.
- **The model cannot add anyone.** On every probe query, every returned person is in the stage-1 candidate set. A person named outside it is dropped by code, and a probe that shows one in the answer fails the gate.

- **No row carries an attributed capability.** Every row’s explanation names a field and a term from the record. A row whose explanation asserts anything about the person that is not in their own words fails the gate.

Failing any of them, the two-stage answer is dropped and the cap stands. An answer that quietly promotes an unplaced person to a confident one, or names someone no record matched, is worse than a short answer — it is the fabrication the abstention rule exists to prevent, arriving by a different route.

Placement and cost

After Amendment 2’s P3.8 work, since it extends it, and therefore after P3.6 and P3.7.

Estimate: 3–4 focused days, plus the cost measurement. The work is in three parts:

- the candidate-set function in code, shared with P3.8’s expansion, recording each candidate’s field and term
- the answer format: the model selects by reference from the candidate set, code drops anything outside it, orders the rows, and renders each from the record with the exclusions stated
- the prompt reduced to selecting from a given set and wording the answer

That is a different piece of work from lifting a cap and adding per-row explanations, which is what a first estimate of 1.5 days sized. It addresses both of the problems that estimate missed. A broad answer no longer has to be written out row by row inside an 8,000-token reply, and the same query yields the same candidate set. That is the part an API contract can honestly guarantee.

This is sizing, not a schedule. The 22 September scope revision dropped day counts and deadlines as pressure on scope or quality, not estimates of how large a piece of work is.

Phase 3 estimate revised from 7.5–9.5 to **10.5–13.5 focused days**.

Cross-cutting rule, restated again

No feature attributes a skill, tool, or capability to a named individual unless that individual’s own words are its source. Occupational descriptions are attributed to occupations. Search behaviour is attributed to the search box. A reason a row appears in a list is attributed to the field that matched.

*Data attribution: this system includes information from the O*NET 30.3 Database by the U.S. Department of Labor, Employment and Training Administration (USDOL/ETA), used under CC BY 4.0. USDOL/ETA has not endorsed this application.*